

Data Intensive Systems

IN BA6

Notes by Ali EL AZDI

February 16, 2026

Introduction

This document is designed to offer a concise, LaTeX-styled overview of the Data Intensive Systems course, with a focus on essential content and clarity. The **ConceptName.** notation is usually used for definitions in that same spirit. The latest version I am currently working on (not necessarily stable) is available at <https://github.com/elazdi-al/notes>. I do make mistakes often, and sharing this document is also a way to have my notes proofread. If you notice anything weird, incorrect, or missing, please do not hesitate to contact me at ali.elazdi@epfl.ch. I would be happy to answer and explain why I made certain choices.

with this being said, welcome back everyone and good luck!

Contents

Contents	3
1 L1 - Introduction	5
1.1 Why data management?	5
1.1.1 What is data?	5
1.1.2 What is a database?	6
1.1.3 What is a Database Management System (DBMS)?	6
1.1.4 What does a DBMS do?	6
1.1.5 Is your file system a DBMS?	6
1.1.6 Is the web a DBMS?	7
1.1.7 The scope of DBMS	7
1.1.8 What is the intellectual contribution?	8
1.2 DBMS Architecture	8
1.2.1 Describing data	8
1.2.2 Example: Schema of a University database	8
1.2.3 Describing data: Levels of abstraction	8
1.2.4 DBMS cares about data independence types	9
1.2.5 Simplified DBMS architecture	9
1.2.6 Relational data model	9
1.2.7 Database design requirements	10
1.2.8 What is a conceptual design	10
1.2.9 Conceptual Design: Entity-Relationship Model	10
1.2.10 Key constraints	12
1.2.11 Participation constraints	12
1.2.12 Weak entities	13
1.2.13 ISA (“is a”) hierarchies	13
1.2.14 Aggregation	14
1.2.15 Conceptual design using the ER model	15
2 L2 - SQL Introduction: Part 1	16
2.1 Relational terminology	16
2.2 Basic SQL queries	16
2.2.1 The simplest query	16
2.2.2 Showing specific columns (Projection)	16

2.2.3	Showing specific rows (Selection)	17
2.2.4	SQL vocabulary	17
2.2.5	Evaluation order	17
2.3	Querying multiple relations	17
2.3.1	Naïve evaluation in steps	18
2.4	Aggregate operators	18
2.4.1	Examples	19
2.4.2	Name and age of the oldest sailor(s)	19
2.5	GROUP BY	19
2.5.1	Using GROUP BY	20
2.5.2	HAVING — filtering groups	20

Chapter 1

L1 - Introduction

1.1 Why data management?

Modern software is data-driven: systems are useful only when data can be stored, shared, and queried correctly over time. This chapter builds that idea step by step, from raw data to DBMS architecture.

1.1.1 What is data?

We first separate *raw facts* from *usable meaning*.

Data. Raw facts used as input for reasoning, discussion, and computation. Data can be useful, irrelevant, or redundant. It must be processed to become meaningful.

Information. Processed data with meaning in context. Information is relevant to a specific problem and actionable for decision-making.

Practical flow:

$$\text{Data} \xrightarrow{\text{organize} + \text{process}} \text{Information}$$

Information then supports higher-level outcomes such as **knowledge** and **wisdom**.

1.1.2 What is a database?

Once data has meaning, we need structure.

Database. A large, integrated, structured collection of data, usually modeling a real-world enterprise.

Entity. An object type we store data about (for example, a student or a course).

Relationship. A meaningful association between entities (for example, a student enrolls in a course).

Example: University. Entities are courses, students, and professors; relationships include enrollment and teaching.

1.1.3 What is a Database Management System (DBMS)?

A database alone is passive. We need software that manages access and correctness.

DBMS. A software system designed to store, manage, and facilitate access to databases.

A useful decomposition is:

$$\text{DBMS} = \text{interrelated data (database)} + \text{programs to access/manage it}$$

1.1.4 What does a DBMS do?

Core role of a DBMS. Provide efficient, reliable, convenient, and safe multi-user storage of and access to massive persistent data.

In practice, this means:

- failure protection against hardware/software faults, power loss, and malicious behavior,
- high throughput (thousands of queries/updates per second),
- high availability (24/7 operation),
- concurrency control for correct multi-user access,
- support for very large-scale data,
- persistence (data outlives programs),
- high-level declarative access with physical data independence.

Persistent data. Data that remains stored beyond the lifetime of a process or program execution.

Concurrency control. Mechanisms that coordinate simultaneous operations so outcomes remain correct.

1.1.5 Is your file system a DBMS?

This thought experiment shows why plain files are often insufficient for data-intensive multi-user applications.

Thought experiment 1: concurrent edits

Two collaborators edit and save the same file at nearly the same time. The natural question is: *whose changes survive?* Outcomes can be yours, your partner's, both, neither, or simply **unclear**.

Thought experiment 2: update during power loss

A file update is in progress and power fails. Which changes survive: all, none, only changes since last save, or again **unclear**?

The key issue is the missing contract: guarantees are unclear, which makes application logic fragile and hard to reason about.

How do you write programs over a platform that promises “???”?

If correctness, durability, isolation, and recovery guarantees are not explicit, every application must re-implement them. **A DBMS exists to provide these guarantees systematically.**

1.1.6 Is the web a DBMS?

The web offers strong information retrieval, but retrieval is not equivalent to database management.

The web is not a DBMS

- data is mostly unstructured and untyped,
- the “correct” answer for many queries is not well-defined,
- completeness is not guaranteed,
- data manipulation guarantees are limited,
- freshness, cross-item consistency, and fault-tolerance guarantees are weaker or undefined.

Practical architecture

Most serious websites place a DBMS behind the web layer (for example, `facebook.com` with MySQL).

Is ChatGPT a DBMS???

Short answer. No. ChatGPT is a language model interface, not a DBMS.

It can connect to database-backed systems, but by itself it does not provide the full DBMS contract (schema-centered storage, transaction guarantees, and complete declarative data management).

1.1.7 The scope of DBMS

The question is no longer only “*where to store bytes*”; it is how to query, update, and trust data under real workload conditions.

What more could we want than a file system?

- simple and efficient *ad-hoc* querying,
- robust concurrency control,
- recovery mechanisms,
- the benefits of well-designed data models.

Ad-hoc query. A query formed for a specific immediate need, not predefined as part of a fixed workflow. *With many queries, users, transactions, and resource constraints, complexity increases quickly. Managing that complexity is a core DBMS concern.*

Can the OS offer services like memory management etc? Again, not really

The operating system is essential, but it does not provide the complete data-management contract that applications need.

1.1.8 What is the intellectual contribution?

DBMS is not only engineering practice; it also contributes key ideas about representation, semantics, and correctness.

Core contributions of DBMS thinking

- **Representation of information:** data modeling,
- **Languages and systems for querying data:** complex queries with real semantics over massive data,
- **Concurrency control for data manipulation:** controlled concurrent access with transactional meaning,
- **Reliable data storage:** preserving semantics even after failures (for example, power loss).

Transactional semantics. The guarantee that concurrent and failing executions still behave like correct transactions under clear correctness rules.

1.2 DBMS Architecture

The main architectural question is simple: how do we keep one coherent meaning of data while serving different users and storing bytes efficiently on disk?

1.2.1 Describing data

Data model. A collection of concepts for describing data.

Data models are high-level abstractions: they hide low-level storage details and let us reason about structure and meaning first. Typical model families include *relational*, *hierarchical*, *graph*, and *object-oriented*.

Relation. A named table with rows (tuples) and columns (attributes).

Schema. The structural definition of a relation: attribute names, types, and organization.

In the **relational model**, data is represented as a set of records organized into relations. In **nested models**, not all data is forced into flat tables; hierarchies and arrays are natural first-class structures.

Schema vs. data. Schema is the type (for example, *string*); data is a value instance (for example, *"hello"*) that follows that type.

1.2.2 Example: Schema of a University database

We use one running example throughout the rest of the section.

Students `sid:string, name:string, login:string, age:integer, gpa:real`

Enrolled `sid:string, cid:string, grade:string`

Courses `cid:string, cname:string, credits:string`

Some later examples use `credits:integer`. The important point is explicit typing in the schema.

1.2.3 Describing data: Levels of abstraction

External schema. User- or application-specific views, often used for access control.

Logical schema (conceptual level). The main global structure seen by users and application programs.

Physical schema (internal level). How data is stored on disk: files, indexes, and storage layout choices.

The mapping is:

External schemas $\rightarrow$ Logical schema $\rightarrow$ Physical schema

Many external views map to one logical schema that is then implemented by physical structures.

Running example across levels

External views (access control)

Students_Info `sid:string, name:string`

Course_Enrollment `cid:string, #enrolled:integer`

Logical schema (what applications see)

The logical level uses the university relations above: **Students**, **Courses**, and **Enrolled** (with `credits` often shown as `integer` in this level).

Physical schema (how storage is organized)

Relations can be stored as unordered files, with indexes such as `Students.sid` and `Courses.cid`.

1.2.4 DBMS cares about data independence types

Data independence. The ability to change the schema at one level of a database system without changing the schema at the next higher level.

Logical data independence. The ability to change the conceptual schema without changing user views.

Physical data independence. The ability to change the internal schema without changing the conceptual schema or user views.

This matters because data-intensive systems evolve constantly: applications should keep working while storage layouts, indexes, and conceptual structures improve over time.

1.2.5 Simplified DBMS architecture

The architecture has two linked flows: a *design flow* to define data and a *query flow* to access data efficiently.

Design flow (from requirements to storage)

- **Conceptual design:** capture the enterprise at a high level, typically with ER models.
- **Logical design:** translate ER structures into a relational model.
- **Physical design:** choose indexing and storage structures.
- **Database storage:** persist data in disk files.

Query flow (from SQL to results)

Applications issue SQL (rooted in relational algebra). Inside the DBMS, processing is layered:

- query optimization and execution,
- relational operators,
- access methods,
- buffer management,
- disk space management.

Cross-cutting services enforce correctness and performance across these layers: query processing, transaction management, and concurrency control.

1.2.6 Relational data model

In practice, relational design relies on rows/columns and explicit links between relations through keys.

Key. An attribute (or set of attributes) whose value identifies a tuple uniquely inside a relation.

Foreign key. An attribute (or set) in one relation whose values reference keys in another relation, linking the two relations.

Example relation instance

	SID	CID	Grade		SID	Name	Login	Age	GPA
Enrolled	53666	Carnatic101	5	Students	53666	Jones	jones@cs	18	5.4
	53666	Raggae203	5.5		53688	Smith	smith@eecs	18	4.2
	53650	Topology112	6		53650	Smith	smith@math	19	4.8
	53666	History105	5						

Here, `Enrolled.SID` acts as a foreign key referencing `Students.SID`.

This raises the next design question: *how do we design such schemas systematically?*

1.2.7 Database design requirements

Database design proceeds in stages, from problem understanding to implementation choices.

- **Requirements analysis:** identify user needs and what the database must support.
- **Conceptual design:** produce a high-level enterprise description (often with ER).
- **Logical design:** translate ER into the DBMS data model.
- **Schema refinement:** improve consistency and normalization.
- **Physical design:** choose indexes and disk layout.
- **Security design:** define who can access what.

1.2.8 What is a conceptual design

Conceptual design. A high-level description of the enterprise data, independent of storage and implementation details.

It answers three foundational questions:

- what are the *entities* and *relationships* in the enterprise?
- what information about them should the database store?
- which *integrity constraints* must always hold?

ER diagram. A pictorial representation of the conceptual schema in the ER model.

An ER diagram can be mapped into a relational schema; this bridge connects conceptual understanding to implementable database design.

1.2.9 Conceptual Design: Entity-Relationship Model

The ER model is the standard language for conceptual design: it captures entities, attributes, relationships, and integrity constraints before logical and physical design decisions.

Basics of ER model

Entity. A real-world object, distinguishable from other objects.

Attribute. A property used to describe an entity in the database.

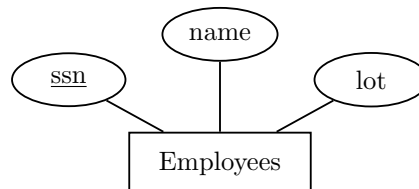
Entity set. A collection of similar entities that share the same attributes.

Domain. The set of allowed values for an attribute.

Key (in ER). An attribute (or minimal attribute set) that uniquely identifies entities in an entity set.

Entity-set schema example

For the entity set **Employees**, we use attributes **ssn**, **name**, and **lot**, with **ssn** as key.



Instance of an entity set. The currently stored collection of entities in that set.

Example instances for **Employees**:

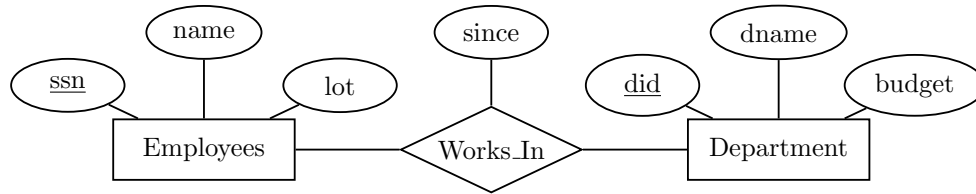
- (123-22-3666, John, P1)
- (231-31-5368, Tom, P2)

Relationships in ER model

Relationship. An association among two or more entities.

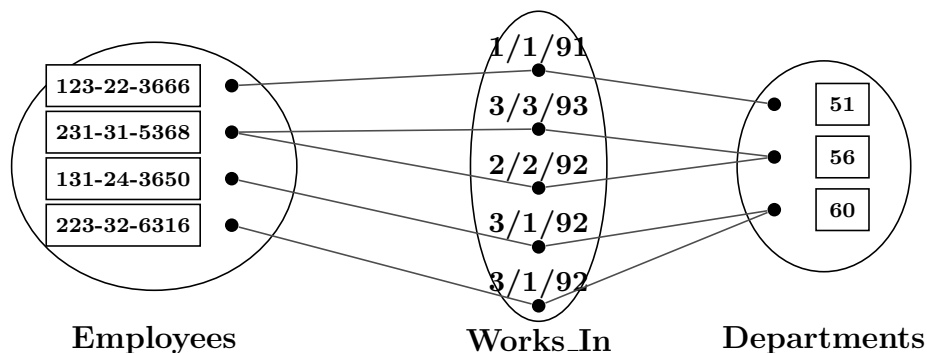
Relationship set. A collection of similar relationships.

Relationships may have their own attributes. Example: **Works_In** links **Employees** and **Department**, with relationship attribute **since**.



Instance of a relationship set. The currently stored set of relationship tuples between participating entity sets.

Example instance of **Works_In(ssn, did, since)** with links to participating entity instances:

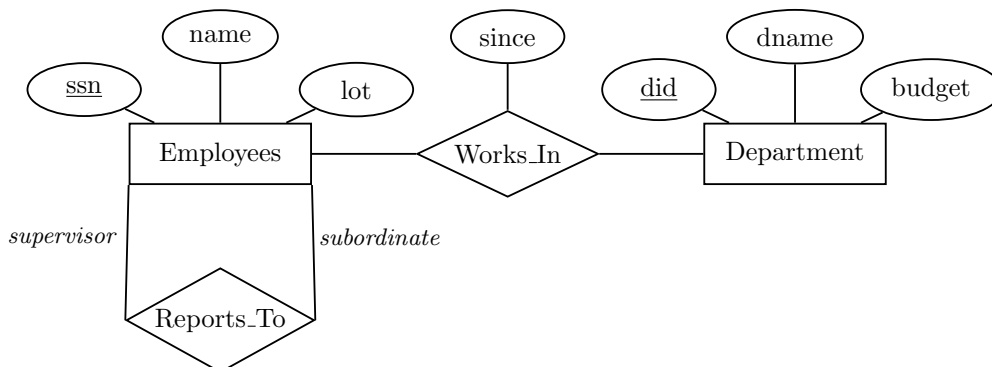


Role names and repeated participation

The same entity set can participate in different relationship sets, and can also appear multiple times in one relationship set under different roles.

Role name. A label identifying the function of an entity set when it appears more than once in the same relationship.

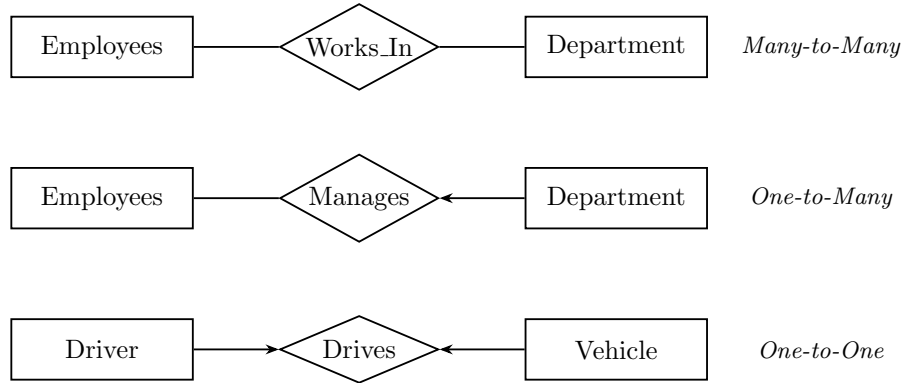
Example: in **Reports_To**, **Employees** appears twice with roles *supervisor* and *subordinate*; the same entity set also participates in other relationships such as **Works_In**.



1.2.10 Key constraints

Key constraints restrict how entities participate in relationships. An arrow from an entity set to a relationship set means each entity participates in *at most one* relationship instance.

Key constraint. A restriction stating that each entity in a set can participate in at most one relationship instance of a given relationship set.



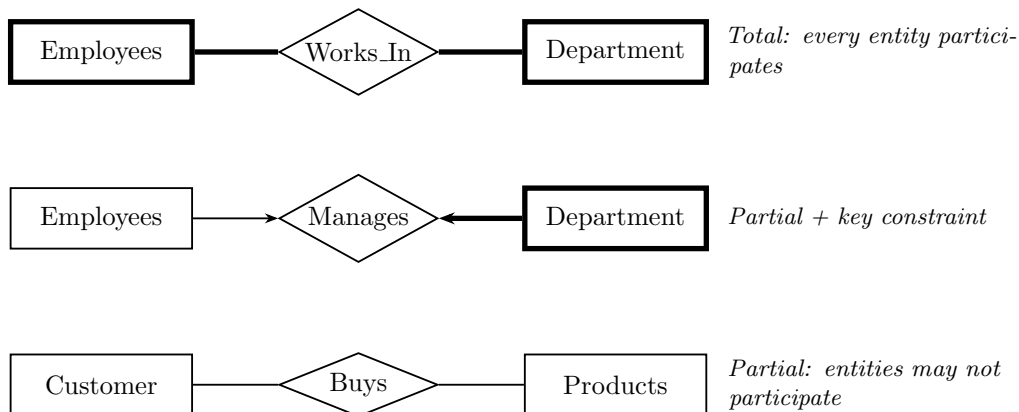
- **Many-to-Many:** no arrows; an employee can work in many departments, and a department can have many employees.
- **One-to-Many:** arrow on one side; each department has at most one manager (key constraint on Manages).
- **One-to-One:** arrows on both sides; each driver drives at most one vehicle and each vehicle has at most one driver.

1.2.11 Participation constraints

Participation constraints specify whether *every* entity in a set must participate in a relationship set.

Total participation. Every entity in the set must participate in at least one relationship instance. Drawn with a **thick line**.

Partial participation. Some entities may not participate in any relationship instance. Drawn with a thin line (default).



Combining key and participation constraints gives precise cardinality semantics:

- thick line + no arrow = total, many: every entity participates, possibly in multiple instances.
- thick line + arrow = total, at most one: every entity participates in exactly one instance.
- thin line + arrow = partial, at most one: an entity participates in zero or one instance.
- thin line + no arrow = partial, many: an entity may or may not participate, with no upper bound.

1.2.12 Weak entities

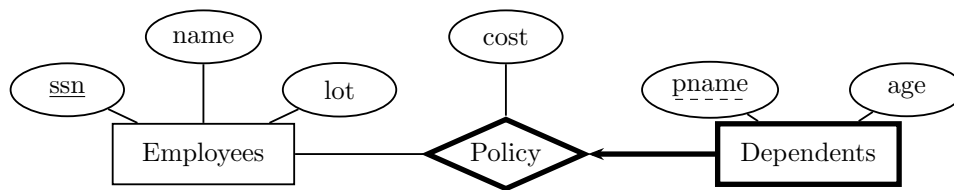
Weak entity. An entity that can be identified uniquely only by considering the primary key of another (*owner*) entity together with its own partial key.

Identifying relationship. The relationship set that links a weak entity set to its owner entity set. The weak entity must have total participation in this relationship.

Rules for weak entities:

- Owner and weak entity set participate in a one-to-many relationship (one owner, many weak entities).
- The weak entity set must have **total participation** in the identifying relationship.
- Weak entities have only a **partial key** (drawn with a dashed underline): it uniquely identifies weak entities only within the scope of a given owner.

Notation: the weak entity and its identifying relationship are drawn with **thick borders**; the arrow from the weak entity to the identifying relationship indicates the key constraint.

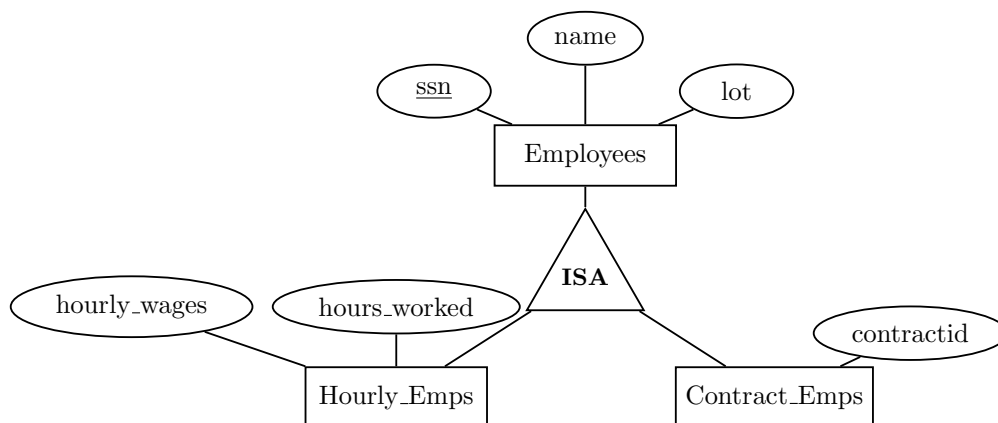


Here, **Dependents** is the weak entity owned by **Employees** through the identifying relationship **Policy**. The attribute **pname** is a partial key.

1.2.13 ISA (“is a”) hierarchies

ISA hierarchy. A relationship between a superclass entity set and one or more subclass entity sets, where each subclass entity “is a” member of the superclass. Attributes are inherited from superclass to subclass.

If we declare A **ISA** B, then every A entity is also considered a B entity and inherits all attributes of B.



Hourly_Emps and **Contract_Emps** are subclasses of **Employees**: they inherit **ssn**, **name**, and **lot**, and add their own specialized attributes.

Two orthogonal design constraints govern ISA hierarchies:

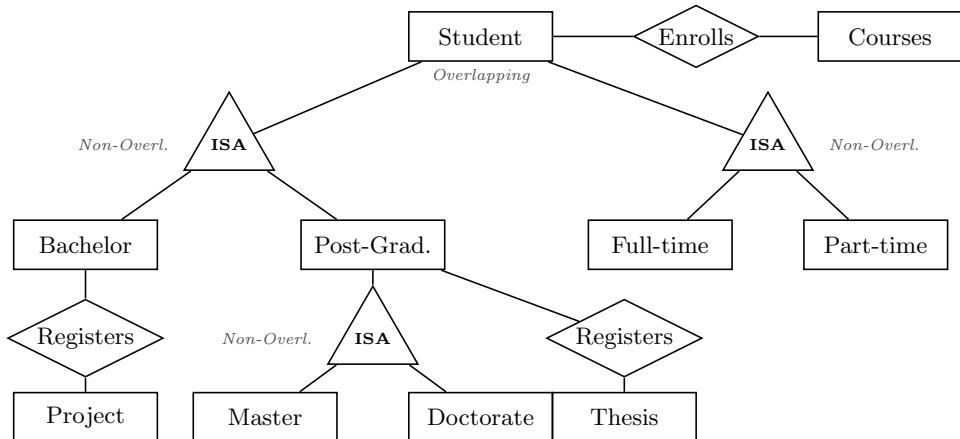
- **Overlap constraint:** can an entity belong to more than one subclass? Default: *no* (disjoint).
- **Covering constraint:** must every superclass entity belong to at least one subclass? Default: *no* (partial coverage).

Reasons to use ISA:

- add descriptive attributes specific to a subclass,
- identify entity sets that participate in specific relationship sets.

ISA example: Student hierarchy

A student can be classified by degree type (Bachelor, Post-Graduate) *and* by enrollment status (Full-time, Part-time) simultaneously. These are two **overlapping** ISA branches from the same superclass. Within each branch, the subclasses are **non-overlapping** (a student is either a Bachelor or a Post-Graduate, not both).



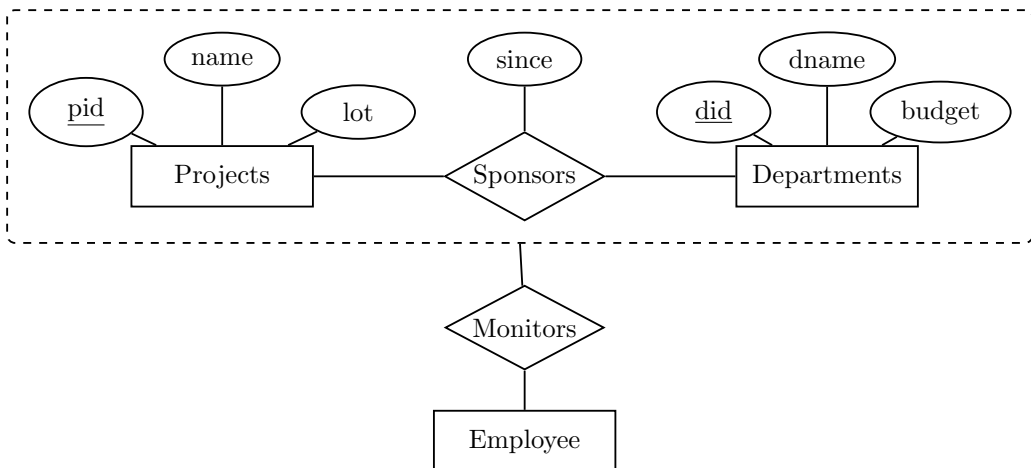
Post-Graduate further specializes into Master and Doctorate (non-overlapping). Each subclass can participate in its own relationship sets: Bachelor students register for Projects; Post-Graduate students register for a Thesis.

1.2.14 Aggregation

Aggregation. A mechanism that treats a relationship set and its participating entity sets as a single higher-level entity set, so that it can participate in other relationships.

Aggregation is needed when we want to model a relationship *involving a relationship set*. For example: departments sponsor projects (the relationship **Sponsors**), and an employee needs to monitor each sponsorship.

Without aggregation we would need a ternary relationship among Projects, Departments, and Employees, which does not capture that monitoring applies to the *sponsorship* itself. With aggregation, the Sponsors relationship is enclosed in a dashed box and treated as an entity set that participates in **Monitors**.



1.2.15 Conceptual design using the ER model

Designing an ER schema involves several recurring choices. The right answer depends on the semantics of the data and the queries the application must support.

Design choices

- Should a concept be modeled as an *entity* or an *attribute*?
- Should a concept be modeled as an *entity* or a *relationship*?
- Should relationships be *binary* or *ternary*? Or use *aggregation*?

Entity vs. attribute

Should *address* be an attribute of Employees or an entity related to Employees?

It depends on how we use the data:

- If there are **several addresses per employee**, address must be an entity (attributes cannot be set-valued in the basic ER model).
- If the **structure** of an address matters (city, street, zip), address must be an entity (attribute values are atomic).

The same reasoning applies to relationship attributes. For example, if **Works_In** has attributes **from** and **to** to record time periods, an employee can work in a department for only one period. If we want to record *several* periods, we must promote the time information into a separate entity (**Duration**) linked by a new relationship.

Entity vs. relationship

Should a concept be a relationship or an entity with relationships to the things it connects? The guideline is: if the concept has **its own identity** or participates in **other relationships**, make it an entity.

Binary vs. ternary relationships

If a ternary relationship can be decomposed into binary relationships without loss of information, prefer binary. If not (some combinations of pairs do not imply the triple), a ternary relationship is needed.

Constraints in the ER model

- A lot of data semantics *can* (and should) be captured in ER diagrams: key constraints, participation constraints, weak entities, ISA hierarchies, and aggregation.
- However, some constraints *cannot* be expressed in ER diagrams (for example, “a manager must also be an employee of the managed department”). These must be enforced by the application or by database-level integrity constraints.

Chapter 2

L2 - SQL Introduction: Part 1

2.1 Relational terminology

Although we already defined the relational model, let's align our vocabulary with SQL:

- **Relation** translates to a **table**.
- **Tuple** translates to a **row**.
- **Attribute** translates to a **column**.

Throughout this chapter, we use the following example **Students** table:

<u>sid</u>	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@ee	18	3.2
53777	White	white@cs	19	4.0

Primary Key (PK). The column (or group of columns) that uniquely identifies a row. Two rows **cannot** share the same PK. Above, sid is the primary key.

2.2 Basic SQL queries

2.2.1 The simplest query

To retrieve all contents of a table, use `SELECT *`:

```
SELECT * FROM Students S
```

`S` is an optional *alias* (range variable) for the **Students** table, useful when referencing columns. This returns the full table shown above.

2.2.2 Showing specific columns (Projection)

To retrieve only certain attributes, replace `*` with column names:

```
SELECT S.name, S.login FROM Students S
```

Projection. Selecting specific columns from a table. The query above projects **name** and **login** from **Students**.

name	login
Jones	jones@cs
Smith	smith@ee
White	white@cs

2.2.3 Showing specific rows (Selection)

To filter rows, use the **WHERE** clause:

```
SELECT * FROM Students S WHERE S.age = 18
```

Selection. Filtering rows based on a condition. The query above selects students whose age equals 18.

<u>sid</u>	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@ee	18	3.2

2.2.4 SQL vocabulary

The general form of a basic SQL query:

```
SELECT [DISTINCT] target-list FROM relation-list WHERE qualification
```

- **relation-list**: a list of table names, each possibly followed by an alias.
- **qualification**: comparisons combined with AND, OR, NOT. Each comparison: *Attr op Const* or *Attr1 op Attr2*.
- **target-list**: a list of attributes from the tables in *relation-list*.
- **DISTINCT**: optional keyword; eliminates duplicate rows. By default, SQL keeps duplicates (the result is a *multiset*).

2.2.5 Evaluation order

Conceptually, a SQL query is evaluated as follows:

1. **FROM**: compute the cross-product of all tables.
2. **WHERE**: discard tuples that fail the condition (*selection*).
3. **SELECT**: keep only the requested columns (*projection*).
4. If **DISTINCT** is specified, eliminate duplicate rows.

2.3 Querying multiple relations

To join two tables, list them both in **FROM** and express the join condition in **WHERE**. Consider the following **Enrolled** table alongside **Students**:

<u>sid</u>	<u>cid</u>	grade
53831	Carnatic101	C
53831	Reggae203	B
53650	Topology112	A
53666	History105	B

“Find all students with grade B”:

```
SELECT S.name, E.cid
FROM Students S, Enrolled E
WHERE S.sid = E.sid AND E.grade = 'B'
```

<u>S.name</u>	<u>E.cid</u>
Jones	History105

2.3.1 Naïve evaluation in steps

The evaluation of the query above proceeds conceptually as follows:

Step 1 – Cross Product (FROM): Combine every row of **Students** with every row of **Enrolled**:

S.sid	S.name	S.login	S.age	S.gpa	E.sid	E.cid	E.grade
53666	Jones	jones@cs	18	3.4	53831	Carnatic101	C
53666	Jones	jones@cs	18	3.4	53831	Reggae203	B
53666	Jones	jones@cs	18	3.4	53650	Topology112	A
53666	Jones	jones@cs	18	3.4	53666	History105	B
53688	Smith	smith@ee	18	3.2	53831	Carnatic101	C
53688	Smith	smith@ee	18	3.2	53831	Reggae203	B
53688	Smith	smith@ee	18	3.2	53650	Topology112	A
53688	Smith	smith@ee	18	3.2	53666	History105	B

Step 2 – Apply the predicate (WHERE): Keep only rows where **S.sid = E.sid** and **E.grade = 'B'**. Only one row survives: Jones (sid 53666) enrolled in History105 with grade B.

Step 3 – Project the requested columns (SELECT): Keep only **S.name** and **E.cid**, yielding the final result: (Jones, History105).

Running example: Sailors, Boats, Reserves

The following three relations are used throughout the rest of this chapter.

Sailors			
<u>sid</u>	sname	rating	age
22	Dustin	7	45.0
31	Lubber	8	55.5
95	Bob	3	63.5

Boats		
<u>bid</u>	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Reserves		
<u>sid</u>	<u>bid</u>	day
22	101	10/10/96
95	103	11/12/96

2.4 Aggregate operators

Aggregate function. A function that operates on a set (or multiset) of values from a column and returns a **single** summary value. Aggregates are a significant extension of relational algebra.

Function	Description
COUNT(*)	number of rows
COUNT([DISTINCT] A)	number of (distinct) non-null values in A
SUM([DISTINCT] A)	sum of (distinct) values
AVG([DISTINCT] A)	average of (distinct) values
MAX(A)	maximum value
MIN(A)	minimum value

2.4.1 Examples

Find the number of sailors:

```
SELECT COUNT(*) FROM Sailors S
```

Find the average age of sailors with rating 10:

```
SELECT AVG(S.age) FROM Sailors S WHERE S.rating = 10
```

Count distinct ratings of sailors named Bob:

```
SELECT COUNT(DISTINCT S.rating) FROM Sailors S WHERE S.sname = 'Bob'
```

2.4.2 Name and age of the oldest sailor(s)

This example illustrates a common pitfall with aggregates.

Incorrect query

```
SELECT S.sname, MAX(S.age) FROM Sailors S
```

This is **wrong**: MAX returns a single value over the entire column, so there is no meaningful way to pair it with a particular `sname`.

Correct approach — subquery

```
SELECT S.sname, S.age
FROM Sailors S
WHERE S.age = (SELECT MAX(S2.age) FROM Sailors S2)
```

The inner query computes the maximum age; the outer query retrieves every sailor matching that value.

Scalar subquery. A subquery that returns exactly one row and one column. It can appear wherever a single value is expected (e.g. in a WHERE comparison).

An equivalent alternative places the subquery on the left:

```
SELECT S.sname, S.age
FROM Sailors S
WHERE (SELECT MAX(S2.age) FROM Sailors S2) = S.age
```

2.5 GROUP BY

So far, aggregates have been applied to *all* qualifying tuples at once. Sometimes we need to apply them to each of several *groups* independently.

Consider: find the age of the youngest sailor for each rating level.

- We don't know in advance how many rating levels exist or what their values are.
- If ratings range from 1 to 10, we *could* write 10 separate queries:

For $i = 1, 2, \dots, 10$:

```
SELECT MIN(S.age) FROM Sailors S WHERE S.rating = i
```

This is obviously impractical.

GROUP BY clause. Partitions tuples into groups based on equal values of the specified columns, then applies aggregate functions independently to each group.

2.5.1 Using GROUP BY

We use an extended `Sailors` table for the following examples:

<u>sid</u>	<u>sname</u>	<u>rating</u>	<u>age</u>
22	Dustin	7	45.0
31	Lubber	8	55.5
95	Bob	3	63.5
52	Smith	3	74.5
68	John	7	56.5
81	Ana	7	71.5

“Find the youngest age for each rating level”:

```
SELECT S.rating, MIN(S.age) FROM Sailors S GROUP BY S.rating
```

The tuples are first partitioned by `rating` (groups: rating 3, rating 7, rating 8), then `MIN(S.age)` is evaluated within each group:

<u>rating</u>	<u>MIN(age)</u>
3	63.5
7	45.0
8	55.5

GROUP BY semantics. The `SELECT` clause may only contain:

- attributes listed in the `GROUP BY` clause, and
- aggregate functions applied to other attributes.

Any non-aggregated column in `SELECT` that is *not* in `GROUP BY` is an error.

2.5.2 HAVING — filtering groups

`HAVING` filters *groups* (after grouping), just as `WHERE` filters *individual tuples* (before grouping).

“Find the youngest age for each rating level that has **at least two** sailors”:

```
SELECT S.rating, MIN(S.age)
FROM Sailors S
GROUP BY S.rating
HAVING COUNT(*) > 1
```

Groups with only one member (rating 8, containing only Lubber) are eliminated:

<u>rating</u>	<u>MIN(age)</u>
3	63.5
7	45.0

HAVING clause. A filter applied *after* grouping. It uses aggregate conditions to keep or discard entire groups, unlike `WHERE` which operates on individual rows before any grouping.